ARGUS: Real-Time Perception-Aware MPC for Aerial Target Tracking with ESDF Obstacle Avoidance

Bruno Pinto

***Abstract*—This paper reports a C++, real-time port of a perception-aware model predictive controller for aerial target tracking, originally implemented in Python with CasADi/IPOPT and known analytical obstacle shapes [1], together with four extensions to its cost and constraint formulation, each evaluated on a common scenario: tracking is moved from a fixed goal point to a band-and-centering formulation that trades reduced obstacle clearance pressure for more frequent loss of visual contact; the field-of-view cost's softplus sharpness is shown to trade a seven-fold drop in peak yaw jerk against command chatter; collision avoidance is moved from a soft cost term to a slacked nonlinear constraint; and an occlusion-aware cost term penalizes obstacles blocking the line of sight to the target, built from an ESDF rebuilt online from simulated onboard sensing instead of known shapes. The deepest result concerns this last term: a derivation showing that the occlusion cost, in its natural point-linearization form, has a gradient that structurally vanishes as the occluding obstacle approaches the target, independent of cost weight. Two candidate fixes, hard-min versus soft-min aggregation of multiple fixed sightline samples, and a steeper penalty function, are derived and tested, and both are shown to leave the underlying vanishing-gradient problem untouched. The root cause is identified as a geometric property of the point-interpolation parametrization itself, not a tuning or sampling artifact, pointing toward an angular or frustum-based formulation as the direction with different sensitivity near the target. The evaluation further splits a single merged reacquisition metric into its two distinct causes, cone exit versus occlusion, showing that the tracking formulation's centering fix resolves the former but leaves the latter, the occlusion cost's own weakness, untouched.**

***Index Terms*—Trajectory Optimization, Model Predictive Control, Occlusion Avoidance, Aerial Target Tracking, Perception-Aware Planning**

I. Introduction

Autonomous aerial tracking of a moving target through cluttered environments requires a controller that keeps the target in view, avoids collisions, and produces a trajectory the vehicle can actually fly. A first version of this controller was built in Python [1], using CasADi and IPOPT as the solver, with obstacles represented as known analytical shapes rather than sensed online.

This paper reports on four extensions made since, each measured on a common evaluation scenario (Section V). I ported the controller to C++, solved in real time with acados [2], and replaced the known analytical obstacle shapes with an ESDF built from simulated onboard sensing. The tracking cost is reformulated from a fixed goal point to a band-and-centering scheme (Section III-A); the field-of-view cost's softplus sharpness is retuned and its effect on command chatter measured (Section III-B); collision avoidance is moved from a soft cost term to a slacked nonlinear constraint (Section III-C); and an occlusion-aware cost term, absent from the original formulation, is added to penalize obstacles blocking the line of sight to the target (Section III-D). The occlusion cost receives the deepest treatment of the four: its gradient is shown to vanish structurally in a specific, common geometric case, and two plausible fixes are tested and shown not to resolve it (Section IV). Section VI states plainly what remains open.

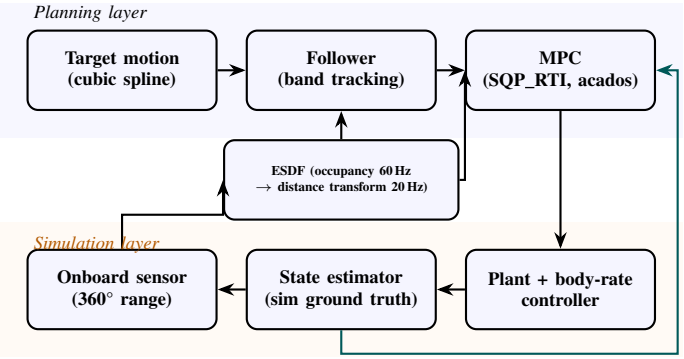


Fig. 1. System architecture. Binary occupancy is rebuilt every sensor sweep (60 Hz); the ESDF is rebuilt once per control tick (20 Hz). The teal path is the state-feedback loop closing the control loop back into the MPC.

II. System Architecture

The system is implemented in C++, generating the MPC solver through acados [2] from a symbolic CasADi model (Fig. 1). The target's trajectory is a known cubic spline, removing target-estimation error as a confound. Each occupancy voxel holds one of three states, free, occupied, or unknown: a tri-state classification, not a probabilistic occupancy grid, since the system does not model sensor or measurement uncertainty.

The sensor and the control loop run on independent clocks: the sensor sweeps at 60 Hz (16.6 ms), the MPC solves at 20 Hz (50 ms), three sweeps per control tick. The occupancy grid is updated every sweep, but the ESDF, the distance transform over that grid, is rebuilt only once per control tick: one build serves all three sweeps inside it. This is a compute-budget decision, since the transform is the expensive step and the MPC's costs only ever read the ESDF, never raw occupancy directly.

The cost is a nonlinear least-squares (NLS) problem: each residual r_i is weighted and squared automatically, cost $=$

$\sum_i W_i r_i^2$. acados approximates the cost Hessian with the Gauss–Newton approximation, $H \approx J^\top W J$, $J = \partial r / \partial p$, built only from each residual's first derivative; the residual's own second-derivative term is dropped entirely. The solver runs in real-time iteration (RTI) mode, one SQP step per control tick, no inner iterations, no line search: a poor local quadratic model is never corrected within the same tick.

The controller extends the perception-aware MPC line of work introduced by Falanga et al. [3], which couples a bearing-centering cost directly into the MPC rather than treating perception as a separate stage. The prior version of this controller [1] already builds on this approach with its own field-of-view cost (Section III); the extensions here, collision avoidance as a constraint and an occlusion-aware cost, go beyond both.

III. MPC Formulation

The back-end solves, at every control tick,

$$\begin{aligned} \min_{x,u,s} \quad & \sum_{k=0}^{N-1} \|r(x_k,u_k)\|^2_W + \|r_N(x_N)\|^2_{W_N} + z_l s + \tfrac{1}{2} Z_l s^2 \\ \text{s.t.} \quad & x_{k+1} = f(x_k,u_k), \\ & c_{\min} \le c_k \le c_{\max}, \quad -\Omega_{\max} \le \Omega_{B,k} \le \Omega_{\max}, \\ & d_{\text{lin}}(p_k) \ge d_{\text{safe}} - s, \quad s \ge 0, \end{aligned} \tag{1}$$

where r stacks the per-term residuals defined below, f is the quadrotor dynamics, c is normalized thrust, Ω_B is body rate, and s is the collision slack (Section III-C). Each subsection below gives one residual's cost, its gradient with respect to the drone's own position (the quantity that matters for how the solver actually behaves under the one-step RTI scheme of Section II), and the issue found with it.

A. Tracking Cost

Two different mechanisms carry tracking authority, used in different rows of Table II, not one combined formula. The *Baseline* rows pursue a single goal point directly: the follower computes it using a trailer analogy, pulled behind the leader at a fixed bearing and distance, turning with the leader the way a towed trailer follows a car around a corner, and the drone's (x, y, z) position is driven toward that point by a plain quadratic residual, $r_{\text{pos}} = p - p_{\text{ref}}$.

The *Band follower* rows zero that position weight (Table I) and replace it with three residuals computed from the drone's position relative to the leader directly, not the trailer point. A distance band leaves the drone free to drift inside a margin at zero cost,

$$r_{\text{dist}} = \max(0, d_{\min} - d) + \max(0, d - d_{\max}), \quad d = \|p - p_{\text{target}}\|. \tag{2}$$

Vertical offset is pinned to a fixed look-down pitch angle θ_b,

$$r_{\text{elev}} = (z - z_{\text{target}}) - d_{\text{horiz}} \tan\theta_b, \tag{3}$$

the relation that follows directly from requiring the camera to pitch down by exactly θ_b at horizontal range d_{horiz}. Which side of the leader the drone orbits on is held by a cosine residual rather than a raw angle difference, so it stays smooth across the $\pm 180^\circ$ wrap,

$$r_{\text{azimuth}} = 1 - \cos(\varphi_{\text{rel}} - \varphi_{\text{target}}). \tag{4}$$

Issue: the Baseline's single fixed point couples tracking error directly to collision and occlusion risk, since nothing in the cost distinguishes a tracking-error point that happens to sit near an obstacle from any other. The Band follower removes that coupling, but then relies on the field-of-view and occlusion terms (Sections III-B–III-D) to keep the target framed, which they only partially do: Table II shows it trading a larger clearance for more lost-contact events, discussed in full in Section V.

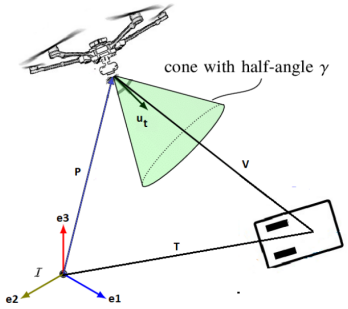


Fig. 2. FOV cone geometry, reproduced from [1]. Its own half-angle label, drawn as γ inside the source figure, is this paper's α, not this paper's own γ (the bearing angle, defined above). $s_{\text{fov}} \ge 0$ exactly when the bearing to the target, V, sits inside the cone.

B. Field-of-View Cost

Let V be the vector from drone to target and $\hat{u}_t$ the camera boresight unit vector, and let γ be the bearing angle between them, $\cos\gamma = V^\top \hat{u}_t / \|V\|$. The target is inside the cone of half-angle α exactly when $\gamma \le \alpha$, written as the hinge argument

$$s_{\text{fov}} = V^\top \hat{u}_t - \|V\| \cos\alpha \ \ge 0, \tag{5}$$

penalized by a squared softplus hinge,

$$r_{\text{fov}} = \frac{1}{\beta} \log\!\left(1 + e^{\beta s_{\text{fov}}}\right). \tag{6}$$

Issue: squaring the raw hinge already makes it continuously differentiable at $s_{\text{fov}} = 0$, but its second derivative still jumps from 0 to 2 there; since the solver's Gauss–Newton Hessian (Section II) depends on curvature, not just gradient, a shallow β leaves this jump active near the operating point and causes command chatter: sharpening β from 5 to 20 drops peak yaw jerk from 42.16 to 6.34 rad/s^2 and eliminates all 40 command sign changes (Appendix, Table IV). Separately, this cost only keeps the target inside the cone, $\gamma \le \alpha$; it does not pull γ toward zero. A centering term, pulling the bearing itself toward dead-center rather than merely keeping it legal, was added once the tracking band (Section III-A) stopped doing so implicitly:

$$r_{\text{center}} = 1 - \cos\gamma. \tag{7}$$

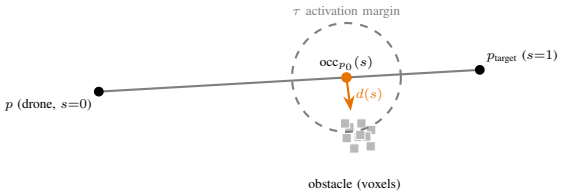


Fig. 3. Occlusion cost geometry. $\text{occ}_{p_0}(s)$ linearizes the ESDF at a point on the drone-to-target sightline; r_{occ} activates once $d(s) < \tau$.

TABLE I
OPERATING WEIGHTS AND JUSTIFICATION.

Residual	Value	Source
r_{pos} (Baseline)	100 / 200	stage/terminal, carried from initial tuning
$r_{\text{dist}}, r_{\text{elev}}, r_{\text{azimuth}}$ (Band)	≈20–100 each	zero in Baseline rows, active only once r_{pos} is zeroed
r_{fov}, β	100, 20	β swept (Table IV)
r_{occ}, τ	20, 1.68 m	τ = FOV width at target range
collision z_l, Z_l	10000, 100	z_l large for exact penalty
rate regularization	15	carried from initial tuning

C. Collision Avoidance

d_{safe} sums the drone radius, one voxel width, and a staleness margin bounding how much the ESDF can change between linearizing it and acting on it, found empirically by logging the worst observed gap between a stale linearization and a fresh query (0.0649 m, rounded to 0.07 m):

$$d_{\text{safe}} = \underbrace{0.15}_{\text{radius}} + \underbrace{0.05}_{\text{voxel}} + \underbrace{0.07}_{\text{staleness}} = 0.27 \text{ m}. \tag{8}$$

Collision avoidance enters (1) as a slacked constraint, $d_{\text{lin}}(p) \geq d_{\text{safe}} - s$, penalized $z_l s + \frac{1}{2} Z_l s^2$.

Issue: as a squared-hinge *cost* (the initial formulation), $r_{\text{obs}} = \max(0, d_{\text{safe}} - d_{\text{lin}})$, the gradient $2W(d_{\text{safe}} - d_{\text{lin}})$ vanishes at the boundary, so no finite weight achieves exact clearance (Appendix, Table III). Reformulating as a constraint fixes this: the slack's linear term has gradient z_l, a constant that does not vanish at $s = 0$, the classical exact-penalty property a quadratic cost never has.

D. Occlusion Cost

Occlusion is costed by linearizing the ESDF at a point along the drone-to-target sightline,

$$\text{occ}_{p_0}(s) = (1-s)\,p + s\,p_{\text{target}}, \qquad r_{\text{occ}} = \max(0,\, \tau - d(s)), \tag{9}$$

with $d(s)$ the ESDF linearized at $\text{occ}_{p_0}(s)$ and τ an activation margin. Differentiating with respect to the drone's own position p,

$$\frac{\partial\, r_{\text{occ}}}{\partial p} = -(1-s)\,\nabla d. \tag{10}$$

Issue: this gradient vanishes structurally as $s \to 1$ (obstruction near the target), independent of weight. Section IV derives why and tests two fixes, neither of which resolves it.

E. Weights

Table I summarizes the operating weights and their source. None were derived analytically; each was set by a sweep (Section III-B, III-C, Appendix) or carried over unchanged once an earlier sweep had already validated it.

IV. Occlusion Cost: Structural Analysis

A. The Vanishing-Gradient Failure Mode

In (10), $(1-s)$ is the entire sensitivity of the occlusion cost to the drone's own position. As the real obstruction approaches the target end of the sightline, $s \to 1$, this factor, and the gradient the drone feels, go to zero regardless of weight. This is exactly the case collision avoidance (Section III-C), which only sees obstacles near the drone, cannot substitute for: the occlusion term is weakest exactly where it is the only term that could help.

Fig. 4 shows this geometry on a real evaluation-suite run: the sightline runs from the drone to the target inside the camera frustum, with own_map's occupied voxels standing in for the ESDF obstacle, occlusion cost off (left) and on (right), same tick.

B. Candidate Fix: Hard-Min vs. Soft-Min Combination

A natural fix costs several fixed sightline fractions instead of one adaptive point, so no sample's $(1-s)$ is itself a function of where the obstruction sits. The hard minimum, $d_{\min}(p) = \min_i d_i(p)$, is non-smooth where two samples tie, and the solver's Gauss–Newton Hessian jumps discretely across that tie, built only from whichever sample currently wins. A soft minimum,

$$d_{\text{soft}}(p) = -\frac{1}{\beta} \log \sum_i e^{-\beta d_i(p)}, \tag{11}$$

blends all samples by softmax weight w_i, giving $\bar{g} = \sum_i w_i (1-s_i) \nabla d_i$, continuous everywhere, recovering the hard case as $\beta \to \infty$. Neither removes the underlying failure: if the obstruction genuinely sits near the target, whichever sample is nearest it still carries a small $(1-s)$ there.

Both were tested on the same scenario, varying only the combination rule (Appendix, Table V): closest approach, field-of-view exceedance, and mean occlusion stayed within a narrow band of each other and of the adaptive single-point baseline. Only solver time moved consistently with β, rising with the curvature term (11) adds near a tie, not with occlusion quality.

C. Candidate Fix: A Steeper Penalty Function

A second candidate reshapes the penalty, $r(d) = 1/d - 1/\tau$, an artificial-potential-field form growing faster as distance shrinks. This changes only the scalar multiplying the gradient direction; it has no effect on $(1-s)$. Multiplying zero by a steeper function of d is still zero. It also reintroduces a singularity at $d = 0$, the same class of problem as an unrelated earlier formula in this project whose derivative diverged at the system's own target operating point and caused a real collision (not detailed further, as it belongs to ongoing work).

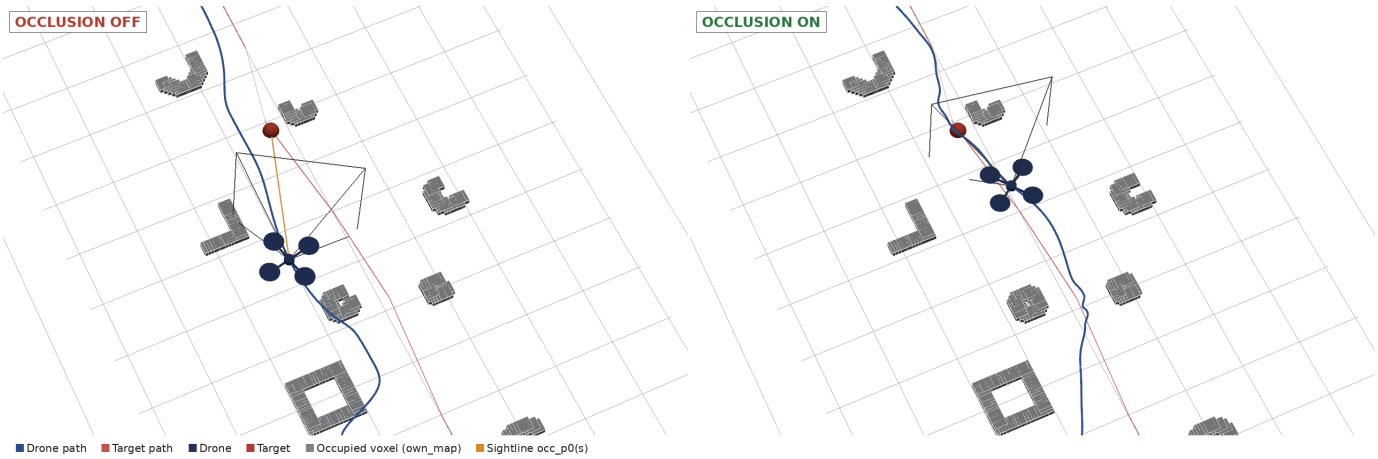


Fig. 4. Third-person technical view, zoomed on the occluded region, occlusion cost off (left) and on (right), same scenario tick as Fig. 8. own_map's real occupied voxels (grey) are drawn in 3-D, not the ESDF's scalar field directly; the camera frustum, drone/target paths, and the sightline $\mathrm{occ}_{p_0}(s)$ from (9) are overlaid.

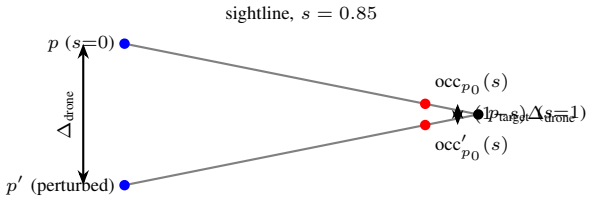


Fig. 5. The lever-arm geometry behind (10). Perturbing the drone from p to p' by Δ_{drone} moves the linearization point $\mathrm{occ}_{p_0}(s)$ by only $(1-s)\Delta_{\text{drone}}$. At $s = 0.85$, shown here, the point moves at 15% of the drone's own displacement; as $s \to 1$ this ratio, and the gradient in (10), goes to zero.

D. The Underlying Geometric Cause

Neither candidate changes the parametrization. $\mathrm{occ}_{p_0}(s)$ is a point on a segment between a moving end (the drone) and a fixed end (the target); a point near the fixed end barely moves when the moving end is perturbed, independent of sampling or aggregation. An obstacle occluding the sightline close to the target is, to first order, not something the drone's own position has leverage over.

Fig. 5 makes this concrete: the same perturbation Δ_{drone} produces a far smaller displacement of the linearization point once s is close to 1, by exactly the factor $(1-s)$ already derived algebraically in (10). No choice of sample fraction, combination rule, or penalty shape changes this ratio, since it is a property of where the point sits on the segment, not of what is evaluated there.

E. Implication for a Working Formulation

What retains sensitivity near the target is the sightline's angle, not a point's position on it: moving the drone swings the sightline's bearing substantially even when the obstruction sits near the target, the same lever-arm effect that makes a point there barely move. This points to an angular or frustum-volume formulation, not a reparametrization of the same point-on-segment idea. It is not built here; Section VI states it as the next step.

V. Results

Table II summarizes the configurations introduced in Sections III-A–III-D. The scenario's obstacle layout and target trajectory are deterministic, so each row is a single run of its configuration, not an average over repeats; the numbers below should be read as one realization each, not as means with an implied variance. Turning the occlusion cost on alone barely moves mean occlusion (63.8% to 62.6%), consistent with Section IV-A: it only has real effect while the obstruction sits closer to the drone than to the target, and the gap is small enough on a single run that it should be treated as indicative, not conclusive, without repeated trials.

The band follower's reacquisition count (4 events, against 0 for the baseline) is the table's largest regression, and it traces directly to Section III-A: the Baseline pins (x, y, z) to the trailer point, so framing is fixed by construction, while the Band follower opens a dead zone on distance, elevation, and azimuth to buy the clearance gain above, and nothing then re-centers the target once the drone uses that freedom to dodge an obstacle, since the field-of-view cost (Section III-B) only penalizes leaving the cone, not drifting toward its edge. The centering term (7) recovers most of this (4 to 1) by rewarding boresight alignment directly, but not all of it: it does nothing for the occlusion cost's own weakness (Section IV-A), and the lost-contact metric counts both failure modes as one event.

Fig. 6 splits that merged count open. The Baseline's occlusion-off row never exceeds the cone; turning its occlusion cost on introduces one brief FOV excursion, not an occlusion-caused loss, at $t \approx 2$ s. The Band follower's four events are three FOV-cone exits plus one occlusion loss, confirming the mechanism above: the centering term's fix (4 to 1) is visible directly as the three red spans disappearing from the bottom panel, with the one orange occlusion span – the failure mode centering does not touch – left unchanged.

TABLE II
MEASURED ON THE FOREST_CLUTTER EVALUATION SCENARIO. REACQ. COUNTS LOST-CONTACT EVENTS (LEAVING THE FOV CONE OR EXCEEDING 90% OCCLUSION).

Configuration	Clearance (m)	Rate max (deg/s)	Occlusion mean/max (%)	Reacq.
Baseline (fixed tracking, occlusion off)	0.30	127.9	63.8 / 84.8	0
Baseline, occlusion cost on	0.35	86.4	62.6 / 82.1	1
Band follower, occlusion on	0.40	60.0	63.2 / 100.0	4
Band follower + centering term	0.38	61.8	65.8 / 100.0	1

Fig. 7 shows one of those three events directly: the same scenario tick, the Band follower's target has drifted past the cone edge while the centering variant, at that same instant, has already pulled it back inside.

No row resolves Section IV's structural problem; each trades a different combination of its symptoms.

Fig. 8 shows the rendered view at the same tick as Fig. 4: obstacle clearance rises from 0.67 m (off) to 1.47 m (on), the same effect Table II reports numerically, without the target leaving the frame in either case at this tick.

VI. LIMITATIONS AND FUTURE WORK

A. Toward an Angular/Frustum Formulation

The direction in Section IV-E, costing angular sensitivity or an approximate frustum-obstacle intersection instead of a linearized point, is not implemented here. It is scoped as the next step: a closed-form cost and gradient well-conditioned under the one-step SQP_RTI solver of Section II, tested against the scenario used throughout Sections IV–V.

B. Remaining Approximation

Even a working angular or frustum formulation would still rely on a local point-or-sphere approximation of the obstacle's surface, from the ESDF's value and gradient at one query location, not a true volumetric intersection. That would require the live occupancy grid inside the differentiable symbolic model the solver is generated from, unavailable at code-generation time in this architecture; an acknowledged limitation of the direction above, not a problem it is expected to resolve.

A related tuning result is worth recording here since it generalizes beyond this one residual: pinning the elevation residual r_{elev} with a nonzero weight bought a narrower altitude band only by raising vertical acceleration, not lowering it, across the evaluation suite, so a tighter constraint was not a smoother one.

C. Evaluation Scope

Every comparison in Section V and the Appendix is internal: configurations of this same controller against each other, on logged scenarios with no repeated trials (Section V). There is no comparison against an external tracking method – a potential-field or sampling-based planner, or a learned policy – so the numbers here establish what the occlusion cost changes relative to this architecture's own baseline, not where this architecture sits relative to the field.

TABLE III
COLLISION COST WEIGHT SWEEP, $d_{\text{SAFE}} = 0.27$ M (SECTION III-C).

Weight	Closest approach (m)
10	0.00
50	0.05
100	0.10
200	0.15
500	0.20

D. Simulation-Only Evaluation

Every number in this paper comes from simulation: the sensor, the dynamics, and the state estimator ("sim ground truth" in Fig. 1) are all simulated, with no sensor noise, wind or aerodynamic disturbance, or communication/actuation latency modeled. The real-time budget of Section II is measured as wall-clock compute only, not as a margin against what a physical vehicle would need once state-estimation error and actuation delay are added on top. None of this architecture has been run on hardware.

VII. CONCLUSION

This paper reported four extensions to a perception-aware MPC for aerial target tracking, ported from Python to C++ with a real-time ESDF replacing known obstacle shapes: a band-and-centering tracking formulation, a retuned field-of-view cost, collision avoidance moved from a cost to a slacked constraint, and an occlusion-aware cost. The first three are measured improvements (Sections III-A–III-C, V): the band-and-centering fix eliminates all three of the evaluation run's FOV-cone-exit events, and sharpening the field-of-view penalty's softplus removes the command chatter a shallow setting leaves behind. The occlusion cost, in its point-linearized form, remains ineffective whenever the obstruction sits near the target rather than the drone, regardless of weight or sightline sampling. Two independent fixes were tested against this failure and both left it unchanged; the cause traces to the geometry of the point-on-segment parametrization itself, not to either candidate's own shortcomings. An angular or frustum-based reformulation, outlined in Section VI, has not yet been implemented or tested.

APPENDIX

A. Evaluation Tooling

Two tools produced the numbers and tables above. A per-run metrics dashboard (Fig. 10) plots clearance, occlusion, bearing offset, tracking error, solver timing, and body-rate smoothness against time for one run, with a pass/fail summary against this

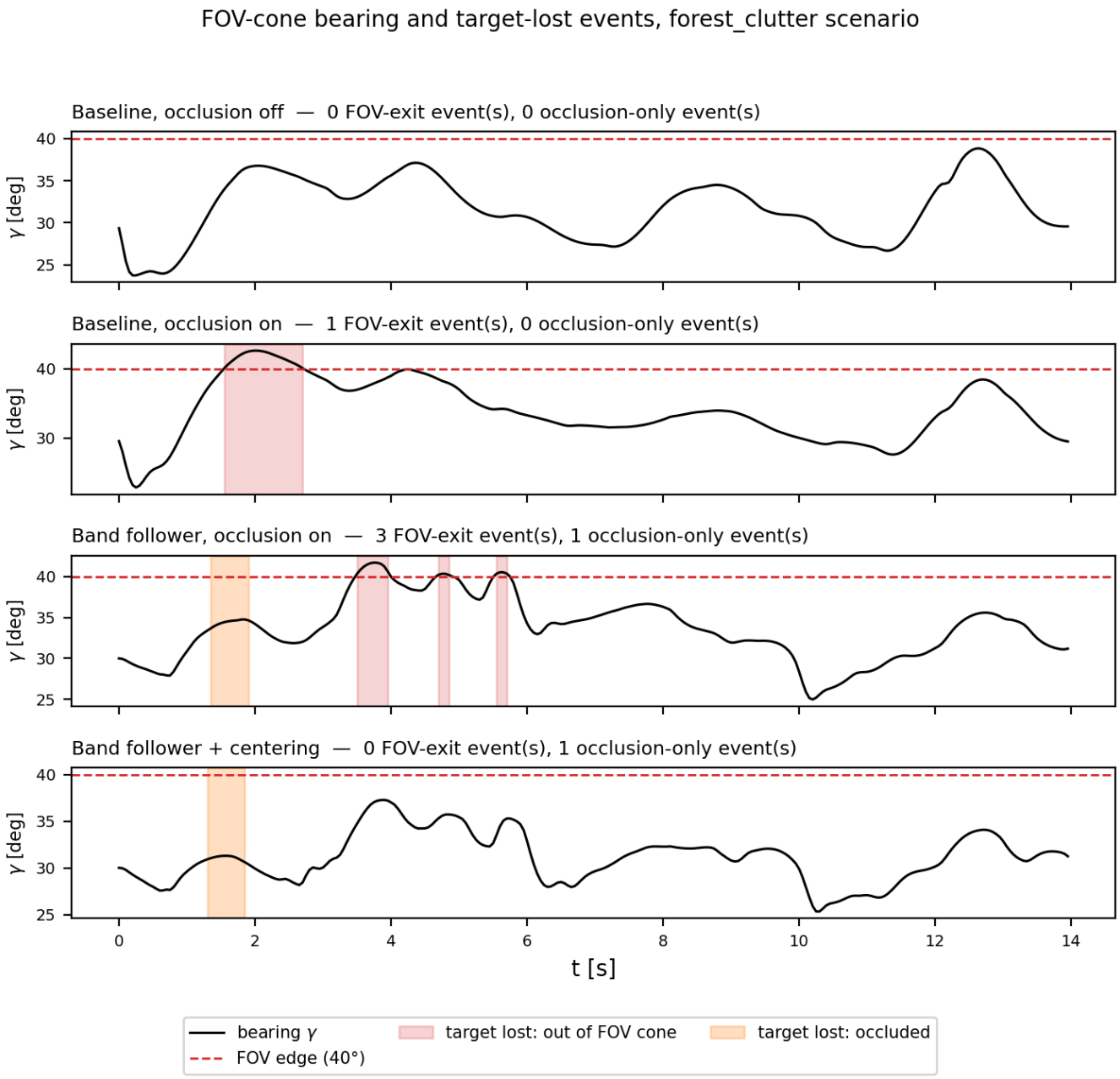


Fig. 6. Combined bearing angle γ against the FOV half-angle threshold, for each row of Table II, with Table II's merged Reacq. count split into its two causes: out of the FOV cone (red) vs. occluded past the lost-contact threshold (orange).

TABLE IV
FOV SOFTPLUS β SWEEP, YAW-CORRECTION SCENARIO (SECTION III-B).

β	FOV exceed. (s)	Peak yaw jerk (rad/s^2)	Sign changes
5	0.30	42.16	40
20	0.30	6.34	0

TABLE V
OCCLUSION COMBINATION RULE COMPARISON (SECTION IV-B).

Rule	Clearance (m)	Occl. mean (%)	Solve time max (ms)
Adaptive (baseline)	0.35	62.6	–
Hard-min	0.30	63.3	2.98
Soft-min, $\beta = 1$	0.30	63.0	2.75
Soft-min, $\beta = 5$	0.30	63.3	4.60
Soft-min, $\beta = 20$	0.30	63.3	5.30

run's own thresholds at the bottom; Fig. 9 isolates one of its panels, the real-time budget claimed in Section II, directly: solve time, ESDF build time, and their sum against the 50 ms deadline, over the same run. A 16-scenario evaluation suite (hand-authored obstacle layouts and target motion profiles, not procedurally generated) runs every configuration change

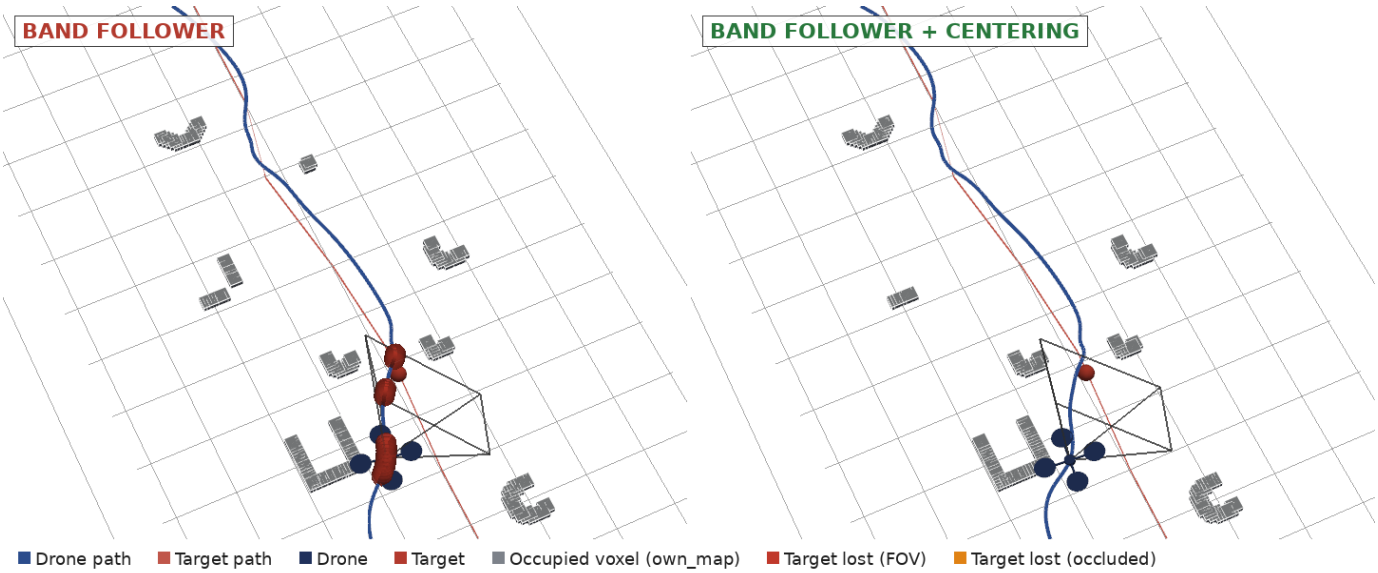


Fig. 7. Same tick ($t = 3.75$ s) and scenario, Band follower (left) vs. Band follower + centering (right) – the first of the three FOV-cone-exit events Fig. 6 shows disappearing. The target marker sits outside the frustum wireframe on the left, inside it on the right; the red path markers are the out-of-FOV mask drawn on the drone's position throughout the run, not just at this tick.

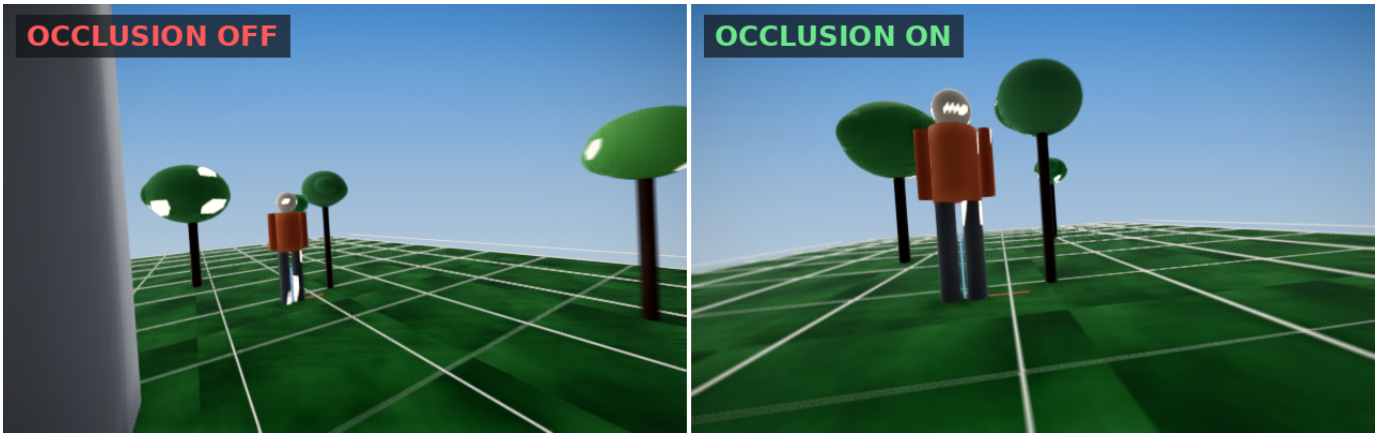


Fig. 8. Rendered onboard camera view, occlusion cost off (left) and on (right), same tick as Fig. 4.

through a common gate set; Fig. 11 is one suite overview grid, a top-down path thumbnail per scenario with a green or red border marking its gate result.

References

[1] B. Pinto, "Autonomous tracking in cluttered environments," instituto Superior Técnico, Universidade de Lisboa, extended abstract.

[2] "acados: fast and embedded solvers for nonlinear optimal control," https://docs.acados.org.

[3] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "PAMPC: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–8.

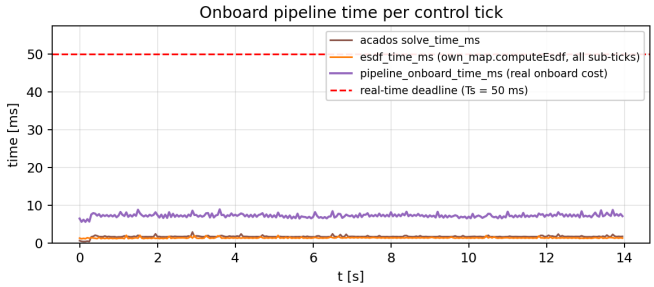
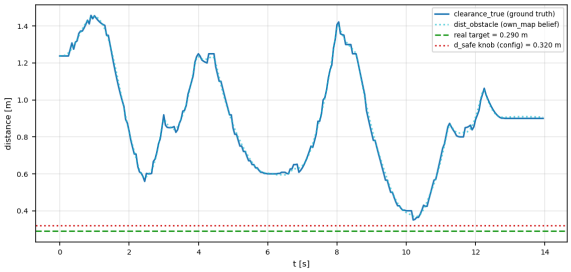


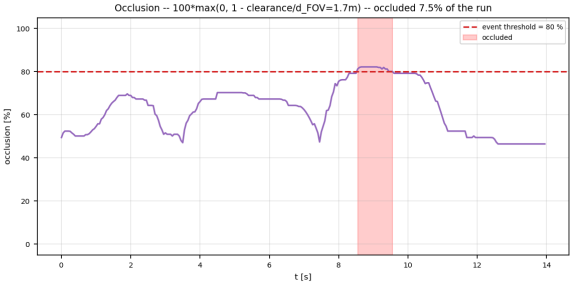
Fig. 9. Onboard pipeline time per control tick, same run as Fig. 10. The solver (acados) and the ESDF rebuild together stay under 9 ms, well inside the 50 ms control-tick deadline (Section II).

Metrics -- logs/online_mapping_trajectory.csv

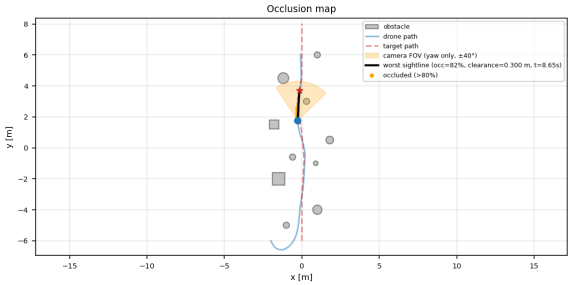
Clearance to nearest obstacle



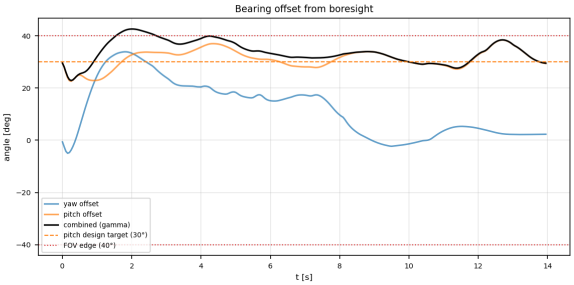
Occlusion -- 100*max(0, 1 - clearance/d_FOV=1.7m) -- occluded 7.5% of the run



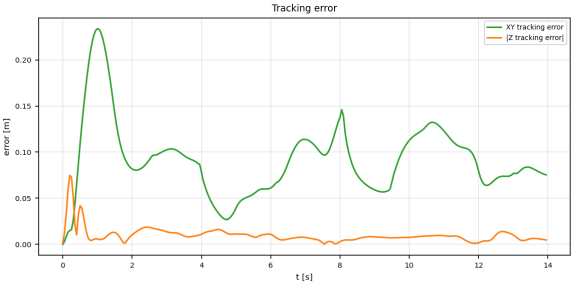
Occlusion map



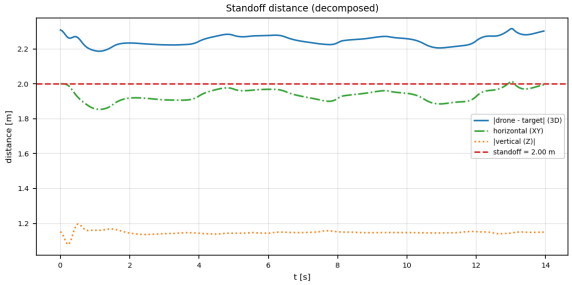
Bearing offset from boresight



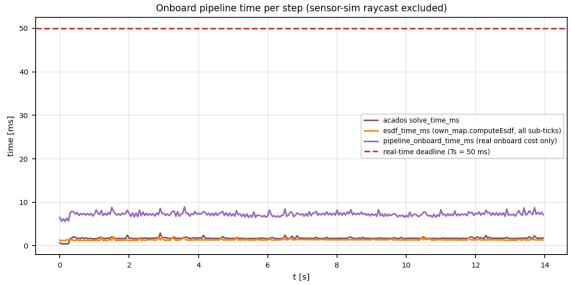
Tracking error



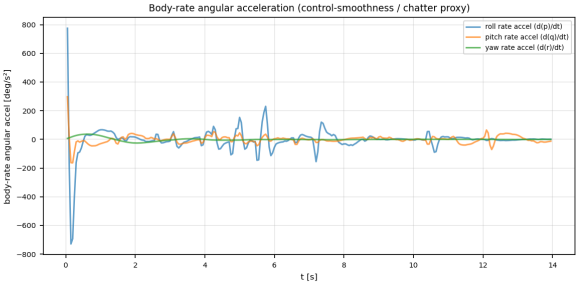
Standoff distance (decomposed)



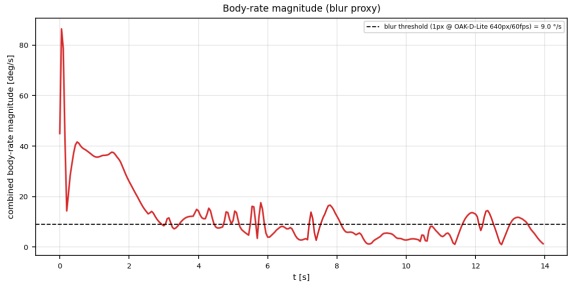
Onboard pipeline time per step (sensor-sim raycast excluded)



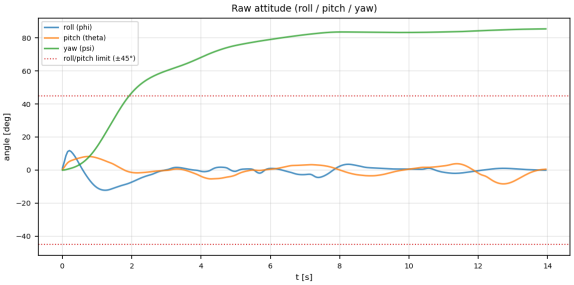
Body-rate angular acceleration (control-smoothness / chatter proxy)



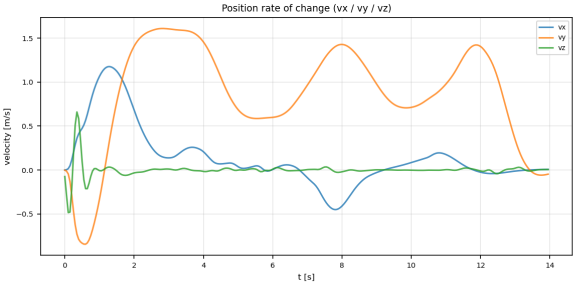
Body-rate magnitude (blur proxy)



Raw attitude (roll / pitch / yaw)



Position rate of change (vx / vy / vz)



Summary (green/red = pass/fail against this run's own thresholds)

Tracking

XY error (mean / max)	0.092 / 0.234 m
Z error (mean / max[.])	-0.007 / 0.075 m
standoff error (mean / max[.])	-0.066 / 0.146 m
path length ratio	1.009

FOV & bearing

yaw offset (mean ± std)	12.033 ± 10.792 °
pitch offset (mean ± std)	31.215 ± 3.609 °
pitch dev. from design	1.21 (max 8.39) °

Collision & safety

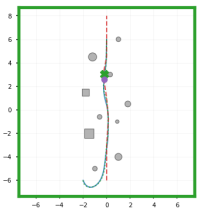
min clearance (true)	0.350 m
real-target violations	**0 steps**
collision free	**True**
clearance margin integral	0.000 m·s

Solver health

solve fail rate	**0.0 %**
solve time (mean / max)	1.728 / 2.930 ms
esdf compute (mean / max)	1.378 / 2.064 ms

Curated scenario suite -- contact sheet (green/red border = hard gates pass/fail)

blur max 128°/s · tracking ±0.04m
forest_clutter



clear 0.30m [collision-free]

trailer ref clear

track 1.93±0.04m reacq 0

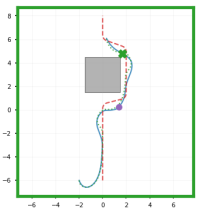
FOV 0% onboard 8.4/12.1ms

blur 13±15 / max 128°/s (thr 9)

target speed 0.00/1.00/1.60 m/s (min/mean/max)

target accel |.| 0.00/0.44/1.07 m/s² (min/mean/max)

blur max 128°/s · tracking ±0.07m
wall_slowdown



clear 0.36m [collision-free]

trailer ref clear

track 1.89±0.07m reacq 0

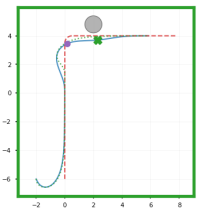
FOV 0% onboard 9.1/14.0ms

blur 22±15 / max 128°/s (thr 9)

target speed 0.03/1.16/1.60 m/s (min/mean/max)

target accel |.| 0.00/0.31/1.21 m/s² (min/mean/max)

blur max 141°/s · tracking ±0.06m
sharp_corner



clear 0.52m [collision-free]

trailer ref clear

track 1.88±0.06m reacq 0

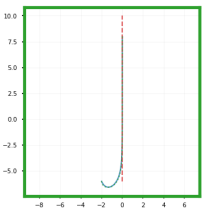
FOV 0% onboard 8.7/10.5ms

blur 21±18 / max 141°/s (thr 9)

target speed 0.03/1.47/1.80 m/s (min/mean/max)

target accel |.| 0.00/0.28/1.20 m/s² (min/mean/max)

blur max 141°/s · tracking ±0.06m
stop_and_go



clear n/a (no obstacles) [collision-free]

n/a (no obstacles)

track 1.92±0.06m reacq 1

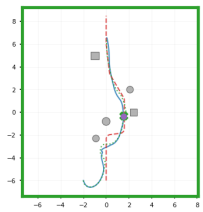
FOV 2% onboard 8.9/12.5ms

blur 13±17 / max 141°/s (thr 9)

target speed 0.00/1.16/1.80 m/s (min/mean/max)

target accel |.| 0.00/0.44/1.80 m/s² (min/mean/max)

blur max 107°/s · tracking ±0.04m
pedestrian_walk



clear 0.60m [collision-free]

trailer ref clear

track 1.92±0.04m reacq 0

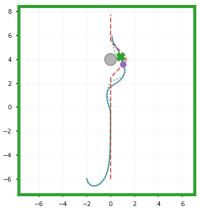
FOV 0% onboard 9.3/17.7ms

blur 14±12 / max 107°/s (thr 9)

target speed 0.00/0.87/1.30 m/s (min/mean/max)

target accel |.| 0.00/0.28/1.63 m/s² (min/mean/max)

blur max 107°/s · tracking ±0.05m
occlusion_cut



clear 0.36m [collision-free]

trailer ref clear

track 1.89±0.05m reacq 0

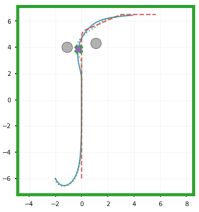
FOV 0% onboard 9.6/18.1ms

blur 16±14 / max 107°/s (thr 9)

target speed 0.02/1.07/1.30 m/s (min/mean/max)

target accel |.| 0.00/0.17/0.87 m/s² (min/mean/max)

blur max 158°/s · tracking ±0.06m
compound_stress



clear 0.45m [collision-free]

trailer ref clear

track 1.87±0.06m reacq 0

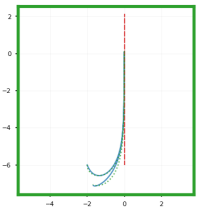
FOV 0% onboard 9.5/12.2ms

blur 24±21 / max 158°/s (thr 9)

target speed 0.03/1.56/1.80 m/s (min/mean/max)

target accel |.| 0.00/0.25/1.40 m/s² (min/mean/max)

blur max 134°/s · tracking ±0.10m
target_reversal



clear n/a (no obstacles) [collision-free]

n/a (no obstacles)

track 1.99±0.10m reacq 0

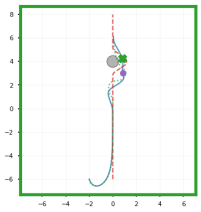
FOV 0% onboard 9.6/17.2ms

blur 15±16 / max 134°/s (thr 9)

target speed 0.00/1.29/1.70 m/s (min/mean/max)

target accel |.| 0.00/0.48/2.64 m/s² (min/mean/max)

blur max 121°/s · tracking ±0.05m
standoff_conflict



clear 0.34m [collision-free]

trailer ref CROSSES obstacle (informational)

track 1.90±0.05m reacq 2

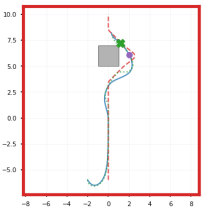
FOV 2% onboard 9.9/30.0ms

blur 24±18 / max 121°/s (thr 9)

target speed 0.02/1.23/1.50 m/s (min/mean/max)

target accel |.| 0.00/0.23/1.00 m/s² (min/mean/max)

blur max 181°/s · FOV exceed 6%
sensor_range_edge



clear 0.32m [collision-free]

trailer ref clear

track 1.86±0.05m reacq 1

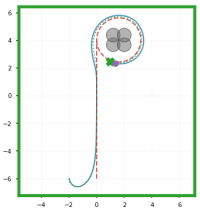
FOV 6% onboard 11.5/36.5ms

blur 31±23 / max 181°/s (thr 9)

target speed 0.04/1.72/2.00 m/s (min/mean/max)

target accel |.| 0.00/0.30/1.67 m/s² (min/mean/max)

blur max 107°/s · tracking ±0.04m
obstacle_roundabout



clear 0.76m [collision-free]

trailer ref clear

track 1.89±0.04m reacq 0

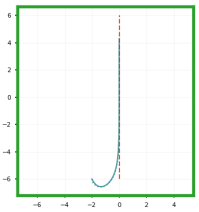
FOV 0% onboard 9.7/19.6ms

blur 26±18 / max 107°/s (thr 9)

target speed 0.02/1.18/1.30 m/s (min/mean/max)

target accel |.| 0.00/0.11/0.90 m/s² (min/mean/max)

blur max 125°/s · FOV exceed 6%
cold_start_acquisition



clear n/a (no obstacles) [collision-free]

n/a (no obstacles)

track 1.90±0.04m reacq 0

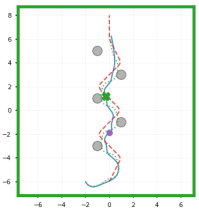
FOV 6% onboard 9.6/17.5ms

blur 18±25 / max 125°/s (thr 9)

target speed 0.03/1.33/1.60 m/s (min/mean/max)

target accel |.| 0.00/0.26/1.07 m/s² (min/mean/max)

blur max 143°/s · tracking ±0.07m
serpentine_clutter



clear 0.30m [collision-free]

trailer ref CROSSES obstacle (informational)

track 1.88±0.07m reacq 4

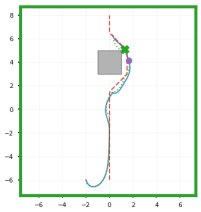
FOV 2% onboard 8.8/19.4ms

blur 33±20 / max 143°/s (thr 9)

target speed 0.02/1.16/1.30 m/s (min/mean/max)

target accel |.| 0.00/0.13/0.90 m/s² (min/mean/max)

blur max 114°/s · tracking ±0.06m
stop_near_obstacle



clear 0.32m [collision-free]

trailer ref clear

track 1.91±0.06m reacq 3

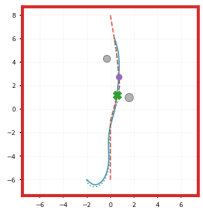
FOV 2% onboard 9.1/14.0ms

blur 20±15 / max 114°/s (thr 9)

target speed 0.00/0.96/1.40 m/s (min/mean/max)

target accel |.| 0.00/0.32/1.40 m/s² (min/mean/max)

blur max 226°/s · FOV exceed 12%
running_person



clear 0.71m [collision-free]

trailer ref clear

track 1.84±0.07m reacq 1

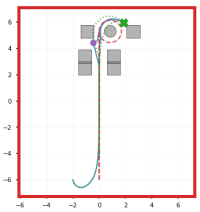
FOV 12% onboard 9.4/15.8ms

blur 28±31 / max 226°/s (thr 9)

target speed 0.00/1.93/2.40 m/s (min/mean/max)

target accel |.| 0.00/0.52/2.40 m/s² (min/mean/max)

blur max 242°/s · tracking ±0.12m
narrow_passage_deep_orbit



clear 0.29m [collision-free]

trailer ref CROSSES obstacle (informational)

track 1.94±0.12m reacq 3

FOV 2% onboard 7.9/12.6ms

blur 32±41 / max 242°/s (thr 9)

target speed 0.00/1.10/1.30 m/s (min/mean/max)

target accel |.| 0.00/0.18/1.69 m/s² (min/mean/max)

drone — target — trailer ref — closest approach (collision-free) — worst occlusion